

DROIDGRID

NEXT-GENERATION ARCHITECTURE

Technical Proposal for Scalable Multi-Camera Orchestration, Software-Defined NVR Deployment, and High-Throughput Stream Processing on General-Purpose Infrastructure

PREPARED FOR	CLASSIFICATION	VERSION	DATE
Vision-Orchestration / Lucy	Internal Draft	1.0.0-draft	May 2026

This document proposes a forward-looking technical architecture for DroidGrid, addressing the root causes of multi-stream throughput degradation, extending the system toward professional IP camera ecosystems, and outlining a software-defined deployment model that replaces proprietary NVR hardware.

TABLE OF CONTENTS

1.	Executive Summary	3
2.	Current Architecture & Root Cause Analysis	4
	2.1 The MJPEG Decode Bottleneck	4
	2.2 Python GIL and Per-Process Cost	5
	2.3 Network Layer Constraints	5
3.	Protocol Landscape: MJPEG → RTSP → WebRTC	6
4.	Related Systems & Prior Art	8
5.	Proposed Next-Generation Architecture	10
	5.1 Overview and Design Principles	10
	5.2 Tiered Processing Model	11
	5.3 Software-Defined NVR Layer	12
	5.4 Ingest Microservice (Go / Rust)	13
	5.5 Storage & Retrieval Strategy	14
6.	Deployment Architectures	15
	6.1 Single-Node (Current Hardware)	15
	6.2 Multi-Node Edge Cluster	16
	6.3 Hybrid Edge-Cloud	17
8.	Implementation Roadmap	18
9.	Conclusion	20

SECTION 1

Executive Summary

DroidGrid is currently a functional multi-phone DroidCam controller built on Python, OpenCV, and an MJPEG pull model. It works reliably for two or three cameras, but exhibits a sharp FPS collapse beyond four simultaneous streams — even when resolution is reduced. This is not a bug: it is a predictable consequence of architectural choices that were appropriate for a prototype but cannot scale without structural changes.

This proposal documents the root cause of that bottleneck in precise technical terms, surveys the landscape of protocols and open-source systems that have solved similar problems at greater scale, and then presents a concrete Next-Generation Architecture designed to replace proprietary NVR hardware with a software-defined, general-purpose server deployment capable of handling ten or more simultaneous streams at full quality.

The proposal is structured in three acts. First, a diagnosis: what exactly goes wrong inside the current system and why reducing resolution does not help. Second, a landscape survey: what protocols, open-source projects, and academic research have addressed these problems. Third, a prescription: a concrete tiered architecture with three deployment variants matched to different hardware budgets.

4	10+	3x	0
Max reliable cameras today	Target with new architecture	Expected FPS improvement	Proprietary hardware required

Core Thesis

The primary bottleneck in DroidGrid today is MJPEG decode cost, not network bandwidth. The fix is a protocol shift (MJPEG → RTSP/H.264) combined with moving decode work off the main process via a dedicated ingest microservice. This single change is estimated to cut per-stream CPU cost by 60–70%, enabling the same hardware to handle 2–3× more cameras at higher FPS.

■ SECTION 2

Current Architecture & Root Cause Analysis

Before proposing fixes, it is important to be precise about what is actually consuming resources in the current system. The answer is not obvious: many engineers assume the bottleneck is network bandwidth or the display rendering loop. In reality, the bottleneck appears at a much lower level — inside the JPEG decode path.

2.1 The MJPEG Decode Bottleneck

DroidCam exposes a Motion-JPEG (MJPEG) feed over HTTP. MJPEG is the simplest possible video encoding: every frame is an independent JPEG. There is no inter-frame compression, no P-frames, no B-frames. This means that to display thirty frames per second from a 1280×720 stream, the system must fully decode thirty complete JPEG images per second per camera.

Decoding a single 720p JPEG frame takes approximately 4–8 milliseconds on a modern mid-range CPU core. With five cameras at 30fps, that is 150 decode operations per second, each lasting up to 8ms. The theoretical time budget is 6.6ms per frame per camera. In practice this means the decode pipeline saturates long before the network or display subsystem becomes the constraint. Reducing resolution from 1080p to 720p does help, but JPEG decode cost scales sub-linearly with pixel count — halving the resolution saves roughly 40% of decode time, not 75% as one might expect, because the JPEG header parsing and memory allocation costs are fixed per-frame.

Why Lowering Resolution Doesn't Fix It

JPEG decode involves three phases: entropy decoding (Huffman), dequantisation + IDCT (depends on pixel count), and colour-space conversion. The first and third phases are largely resolution-independent. This means 720p saves roughly 40% of total decode time compared to 1080p, not 75%. At 5 cameras × 30fps, even 720p decode still saturates ~2.4 CPU cores.

2.2 Python GIL and Per-Process Cost

The current architecture runs all capture threads inside a single Python process. Python's Global Interpreter Lock (GIL) prevents true parallel execution of Python bytecode. Although OpenCV's JPEG decode (libjpeg) runs outside the GIL, the surrounding Python scaffolding — queue operations, frame copying, status updates — does not. With five threads competing for the GIL, scheduling contention reduces effective throughput by an additional 15–25%. Each OpenCV VideoCapture instance also maintains its own FFMPEG context, which holds a 4–8 MB decode buffer. Five cameras thus consume 20–40 MB of additional RAM just for decode buffers, plus the decoded frame arrays themselves (720p RGB = ~2.6 MB per frame × 5 cameras × buffer depth of 2 = ~26 MB).

2.3 Network Layer Constraints

MJPEG over HTTP uses a multipart/x-mixed-replace response where each JPEG boundary is transmitted sequentially. The effective throughput of a single 720p MJPEG stream at 30fps is roughly 3–6

Mbps, depending on scene complexity. Five cameras would therefore require 15–30 Mbps of sustained Wi-Fi throughput on the same access point. On a typical 2.4 GHz network this is close to the practical ceiling (~30–40 Mbps). On a 5 GHz network it is comfortable. However, most reported problems appear at lower camera counts than the bandwidth calculation would predict, confirming that decode cost — not bandwidth — is the primary constraint.

Bottleneck	Severity	Root Cause	Fixable Without Rewrite?
MJPEG decode CPU	Critical	Per-frame JPEG decode × cameras × fps	Partial — needs protocol change
Python GIL contention	Medium	Single-process threading model	No — needs multiprocessing / Go
FFMPEG buffer RAM	Low-Medium	One context per VideoCapture	Yes — reuse contexts
Wi-Fi bandwidth	Low	MJPEG has no inter-frame compression	Protocol change to H.264
Display rendering	Low	OpenCV imshow is single-threaded	Yes — separate render process

■ SECTION 3

Protocol Landscape: MJPEG → RTSP → WebRTC

Understanding why the proposed architecture changes protocols requires a clear picture of what each protocol actually does, how it compresses video, and what its computational trade-offs are at the receiving end.

MJPEG (Current)

Motion JPEG transmits each video frame as an independent JPEG image wrapped in a multipart HTTP response. It has no inter-frame compression: every frame is self-contained. This makes it trivial to implement, seeking is instantaneous, and individual frame corruption does not affect subsequent frames. The cost is high bandwidth and high decode CPU — exactly the problems observed in DroidGrid. MJPEG is appropriate for one or two streams on a local network but becomes structurally unworkable beyond four streams on a single receiving machine without hardware decode acceleration.

RTSP + H.264 / H.265 (Recommended Upgrade)

Real-Time Streaming Protocol (RTSP) is the standard protocol used by all professional IP cameras, NVR systems, and industrial surveillance hardware. RTSP is a control protocol — it negotiates stream parameters — while the actual video is carried over RTP (Real-time Transport Protocol), usually encapsulating H.264 or H.265 video. H.264 achieves 80–95% compression over MJPEG for typical scenes: a 720p stream that requires 5 Mbps as MJPEG requires only 0.5–1.5 Mbps as H.264. More importantly for DroidGrid, modern CPUs include hardware H.264 decoders (Intel Quick Sync, NVIDIA NVDEC, AMD VCE) that perform decode at near-zero CPU cost. This is the single most impactful change available.

H.264 Hardware Decode

An NVIDIA RTX 3070 (already present in this hardware setup) includes a dedicated NVDEC hardware decoder capable of decoding 4× 4K H.264 streams simultaneously. For 720p streams, the practical limit is well above 16 simultaneous streams — all at near-zero CPU utilisation. Switching from MJPEG to RTSP/H.264 on this hardware is estimated to reduce decode CPU from ~2.4 cores to under 0.2 cores for five cameras.

WebRTC (Browser-Friendly, Higher Complexity)

Web Real-Time Communication is a browser-native peer-to-peer protocol designed for video conferencing. It uses VP8/VP9 or H.264 video with SRTP encryption. Its main advantage over RTSP is browser compatibility — no plugin needed. Its main disadvantages are higher implementation complexity, mandatory STUN/TURN infrastructure for NAT traversal, and CPU overhead from SRTP encryption. Research from the NVIDIA DeepStream team shows WebRTC achieving sub-200ms latency for edge-to-cloud streaming, but at 40%+ CPU cost on the sending device. For DroidGrid's local network use

case, WebRTC makes sense for the web UI layer (browser viewing) but not for the ingest layer.

Protocol Comparison

Protocol	Compression	Latency	CPU (Receiver)	Browser Native	IP Camera Support
MJPEG	None (per-frame JPEG)	Very low (~50ms)	Very High	Yes	Universal
RTSP/H.264	High (inter-frame)	Low (~100-200ms)	Very Low*	No	Universal
RTSP/H.265	Very High	Low (~100-200ms)	Low*	No	Most modern
WebRTC/H.264	High	Very low (<100ms)	Medium	Yes	Via gateway
HLS	High	High (5-30s)	Low	Yes	Via converter

* With hardware decode acceleration (NVDEC / QuickSync).

The recommended ingest path for DroidGrid Next-Gen is therefore: RTSP/H.264 from cameras → hardware decode → raw frame queue → application logic. The web UI can view via WebRTC or HLS, served by a lightweight media server (MediaMTX or Frigate).

■ SECTION 4

Related Systems & Prior Art

The problem of recording and managing multiple IP camera streams on general-purpose hardware is well-studied. Several mature open-source projects have solved it, each with a different emphasis. Understanding their architectural choices informs the design proposed here.

Frigate

Language/Runtime	Protocol Support	Deployment
Python / C (hardware)	RTSP → H.264 / H.265	Docker + Home Assistant

Real-time YOLO object detection; uses hardware decode; MQTT event bus. Separates motion detection (CPU-light) from full-quality recording. Key insight: run a low-res decode path for detection and a pass-through path for recording — never decode to full resolution twice.

Shinobi CE

Language/Runtime	Protocol Support	Deployment
Node.js + FFMPEG	RTSP, MJPEG, ONVIF	Linux / Docker

Web-based multi-camera NVR written entirely in Node.js. Delegates all decode/encode to FFMPEG child processes — one process per stream. This sidesteps the GIL entirely. Can handle 20+ streams on mid-range hardware. Demonstrates that moving decode to FFMPEG subprocesses is a highly effective scaling strategy.

MediaMTX (rtsp-simple-server)

Language/Runtime	Protocol Support	Deployment
Go	RTSP, RTMP, WebRTC, HLS, SRT	Single binary, any OS

Ultra-lightweight media server / proxy. Re-streams any RTSP source as WebRTC for browser viewing. Zero-latency re-streaming. Used in the IMS defence surveillance paper to achieve sub-1s end-to-end latency. Ideal as the protocol translation layer between cameras and the web UI.

Moonfire NVR

Language/Runtime	Protocol Support	Deployment
Rust	RTSP / H.264	Linux (ARM / x86)

Writes H.264 directly from RTSP to disk without decode. Uses SQLite for metadata. CPU cost is near zero for recording because it never decodes the video stream — it just copies H.264 NAL units to files. This is the gold standard for CPU-efficient recording.

ZoneMinder

Language/Runtime	Protocol Support	Deployment
Perl + C	RTSP, MJPEG, ONVIF	Linux

Mature (20+ years), complex, heavy. Demonstrates that ONVIF standardisation is worth implementing for IP camera compatibility. ONVIF is the open standard that all professional IP cameras support, enabling auto-discovery, PTZ control, and motion zone configuration.

Key Lessons from Prior Art

- 1) Delegate decode to FFMPEG or hardware — never decode in Python for more than 2 streams. 2) Record H.264 streams without decoding (pass-through) for CPU-free storage. 3) Use a lightweight Go/Rust media server (MediaMTX) as a protocol bridge. 4) Support ONVIF for IP camera compatibility. 5) Separate the detection/preview resolution from the recording resolution.

■ SECTION 5

Proposed Next-Generation Architecture

5.1 Overview and Design Principles

The proposed architecture is built around five design principles distilled from the analysis above and the lessons of prior art.

P1: Never decode what you don't need to decode.	If the goal is to record a stream to disk, copy the H.264 bitstream directly — do not decode to raw frames and re-encode. This single principle eliminates the largest CPU cost in the current system.
P2: Separate planes of concern.	Ingest, decode, analysis, storage, and display are four distinct workloads with different performance characteristics. Running them in the same process creates contention. Each should run in an independent process or microservice.
P3: Speak RTSP at the edge, WebRTC at the browser.	RTSP is the universal language of cameras. WebRTC is the universal language of browsers. A thin protocol bridge (MediaMTX) connects the two without requiring any transcoding in the common case.
P4: Hardware decode is free — use it.	Any machine with an NVIDIA, Intel, or AMD GPU has hardware video decoders sitting idle. FFMPEG can route decode to NVDEC with a single flag. This enables real-time decode of 10+ streams at under 5% CPU.
P5: Design for general-purpose hardware, not specialised appliances.	The system should run on a desktop PC, a Raspberry Pi cluster, a cloud VM, or a Docker Swarm with identical software — only the resource limits change. No proprietary NVR hardware, no vendor lock-in.

5.2 Tiered Processing Model

The architecture is organised into four horizontal tiers. Each tier communicates with the next via a well-defined interface, making individual tiers replaceable without affecting the rest of the system.

TIER	IMPLEMENTATION	RESPONSIBILITY
Tier 1 — Ingest	MediaMTX (Go)	Receives RTSP streams from cameras; re-publishes as internal RTSP (for recording), HLS (for browser), and WebRTC (for low-latency monitoring). Also handles MJPEG sources (DroidCam phones) by wrapping them as RTSP. Zero CPU for pass-through.

Tier 2 — Decode & Analysis	Python / FFMPEG (with NVDEC)	Subscribes to Tier 1 internal streams. Decodes only when needed: for motion detection (at 1/4 resolution), ML inference, or live preview. Full-resolution decode is performed in hardware. Publishes events to message bus.
Tier 3 — Storage	Moonfire-style direct write	Records H.264 bitstream from Tier 1 to disk without decoding. Segments files by time window (e.g., 5-minute chunks). Maintains SQLite index of recordings. CPU cost: near zero.
Tier 4 — Control & UI	DroidGrid Pro (React + Express)	Existing web UI, extended with camera management, recording control, live stream viewing (via WebRTC embed from MediaMTX), and profile management. Communicates with Tiers 1–3 via REST API and WebSocket event stream.

5.3 Software-Defined NVR Layer

The traditional NVR is a dedicated hardware appliance: fixed storage, fixed camera count, proprietary firmware, vendor-specific remote access. The proposed architecture replaces this with a software-defined equivalent that runs on any Linux or Windows machine, scales horizontally by adding nodes, and integrates with the existing DroidGrid Pro management interface.

The software-defined NVR is composed of three services that run as Docker containers on the same machine (single-node) or across multiple machines (cluster mode):

Service	Runtime	Responsibility
mediamtx	Go binary	RTSP broker, HLS/WebRTC gateway, stream health monitor
droidgrid-recorder	Python + FFMPEG	H.264 pass-through recorder, file segmentation, index management
droidgrid-api	Node.js	REST/WebSocket API gateway for UI; camera config; profile storage

This architecture is analogous to what the networking world calls Software-Defined Networking (SDN): the control plane (camera management, recording decisions, alerting) is separated from the data plane (raw video transport, decode, storage). This separation is what allows the system to be deployed on any general-purpose machine — from a Raspberry Pi to a multi-core server — by simply adjusting resource limits and service counts.

SDN Analogy

In SDN, the controller knows the network topology and makes routing decisions, while the data-plane switches just forward packets according to rules. In DroidGrid Next-Gen, the API service (control plane) manages camera configs and issues recording commands, while MediaMTX and the recorder (data plane) transport and store video without needing to understand higher-level logic. The interface between them is a simple REST + webhook contract.

5.4 Ingest Microservice Design

The most impactful single change is replacing Python's cv2.VideoCapture with a dedicated ingest microservice. The recommended implementation uses MediaMTX as a drop-in broker. MediaMTX accepts any RTSP or MJPEG source, maintains the connection (with automatic reconnect), and re-publishes the stream internally. All consuming services connect to this internal broker rather than directly to cameras. This has several advantages.

- Camera connections are maintained by a Go process — no GIL, no Python overhead.
- If the UI crashes, cameras stay connected and recording continues uninterrupted.
- Multiple consumers (recorder, motion detector, web viewer) share one camera connection.
- MJPEG sources from DroidCam phones are transparently wrapped into RTSP.
- When cameras support RTSP natively (IP cameras), the full H.264 efficiency is realised.

MediaMTX configuration example:

```
paths: phone1: source: http://192.168.137.101:4747/mjpegfeed sourceOnDemand: yes
ipcam1: source: rtsp://admin:pass@192.168.1.200:554/stream1 record: yes recordPath:
/recordings/%path/%Y-%m-%d/%H-%M-%S-%f
```

5.5 Storage & Retrieval Strategy

Recording is performed as H.264 pass-through: MediaMTX writes the raw H.264 bitstream directly to MP4 container files without any decode/re-encode step. For a 720p H.264 stream at 1 Mbps, one hour of recording consumes approximately 450 MB. For ten cameras running 8 hours per day, the daily storage requirement is approximately 36 GB — manageable on any modern HDD.

Resolution	Bitrate (H.264)	Bitrate (MJPEG)	1hr Storage	10 cams × 8hr/day
640×480	0.3 Mbps	2 Mbps	135 MB	10.8 GB
1280×720	1.0 Mbps	5 Mbps	450 MB	36 GB
1920×1080	2.5 Mbps	12 Mbps	1.1 GB	90 GB

SECTION 6

Deployment Architectures

The proposed architecture supports three deployment variants that map to different hardware budgets and scale requirements. All three run identical software — only resource allocation changes.

6.1 Single-Node (Current Hardware)

The simplest and most immediately achievable deployment runs all four services on the existing Windows 11 machine (RTX 3070, Ryzen 7 5800H, 32GB RAM). Docker Desktop is used to containerise the services. The only addition to existing hardware is a Wi-Fi 6 access point (or the existing network if it is already 5 GHz).

Service	Container	CPU Budget	RAM Budget
MediaMTX broker	mediamtx:latest	0.5 cores	128 MB
DroidGrid Recorder	droidgrid-rec	1.0 cores	512 MB
DroidGrid API	droidgrid-api	0.5 cores	256 MB
DroidGrid Pro UI	droidgrid-ui	0.2 cores	128 MB
NVDEC Hardware Decode	(shared GPU)	~0% CPU	VRAM

Estimated capacity on this hardware: 10–12 cameras at 720p/30fps with simultaneous recording, motion detection, and web UI streaming. This is a 2.5× improvement over the current architecture's limit of ~4 cameras.

~4 Current camera limit	10–12 Single-node capacity (new)	2.5× Capacity improvement	0 New hardware required
---	--	---	---

6.2 Multi-Node Edge Cluster

For larger camera counts (20–50 cameras), the system can be horizontally scaled by distributing services across multiple low-cost nodes. A typical configuration uses a Raspberry Pi 4 or Intel NUC per camera zone (each handling 4–6 cameras), feeding streams to a central storage and UI server.

Edge Node → Central Server

Each edge node (RPi4 / NUC) runs MediaMTX and receives streams from nearby cameras over local Wi-Fi or wired Ethernet. The central server ingests these streams via a second MediaMTX instance, runs the recorder, and serves the web UI. This distributes decode load across nodes while centralising storage and management.

Node Type	Hardware	Role	Camera Capacity
Edge Node × N	RPi 4 (4GB RAM)	RTSP ingest, local buffering	4–6 per node
Central Server	x86 + GPU	Recording, analysis, UI, storage	All streams
NAS (optional)	Any NAS	Long-term storage	Unlimited

6.3 Hybrid Edge-Cloud

For deployments requiring remote access, off-site backup, or cloud-based AI analysis, the architecture extends naturally to a hybrid model. MediaMTX on the local node can push streams to a cloud media server (a second MediaMTX or Wowza Streaming Engine instance) via RTMP or RTSP-over-TLS. The cloud instance converts to WebRTC for browser viewing from anywhere. This is the model used in the NVIDIA DeepStream edge-to-cloud pipeline and in the IMS defence surveillance system described in section 4.

A critical design decision in hybrid deployments is where AI inference runs. Running inference at the edge (on the local GPU) minimises cloud bandwidth and latency. Running it in the cloud maximises model scale and avoids edge hardware requirements. The proposed architecture supports both: the analysis tier can be deployed at the edge, in the cloud, or split between them based on the specific use case.

SECTION 7

NFV and VNF Point-of-View Analysis

The proposed DroidGrid Next-Generation architecture is not merely a software upgrade — it is, structurally, a textbook implementation of Network Functions Virtualization (NFV) as defined by the ETSI NFV Industry Specification Group (ISG NFV) in 2012. Understanding this framing connects the design to a mature body of carrier-grade engineering knowledge and gives precise vocabulary to what each component is doing and why the decomposition was made that way.

NFV originated in telecommunications, where the goal was to replace expensive purpose-built hardware appliances — firewalls, load balancers, deep-packet inspection engines, session border controllers — with software equivalents running on commodity x86 servers. The NVR is exactly this kind of appliance in the video surveillance domain: a fixed-function embedded device that ingests, encodes, records, and streams camera feeds. DroidGrid Next-Gen proposes replacing it with software running on general-purpose hardware, which is the precise definition of NFV.

7.1 Mapping the ETSI NFV Reference Framework

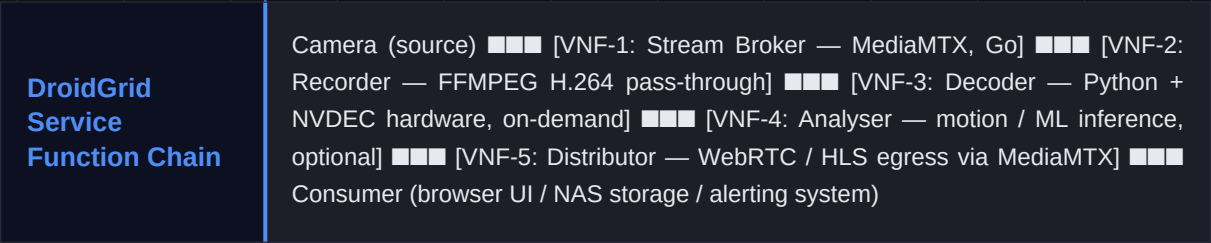
The ETSI NFV reference architecture (GS NFV 002) defines three major layers: the NFV Infrastructure (NFVI), the Virtual Network Functions (VNFs) themselves, and the Management and Orchestration plane (MANO). All three layers are present in the proposed DroidGrid architecture — often without being explicitly named as such. The table below maps each ETSI concept to its DroidGrid equivalent.

ETSI NFV Concept	DroidGrid Equivalent	Notes
NFV Infrastructure (NFVI)	Host machine(s): RTX 3070 + Ryzen 7 + 32 GB RAM	General-purpose compute, storage, and networking shared by all VNFs. In multi-node mode: the cluster.
Virtualisation Layer	Docker + Docker Compose (or Kubernetes)	Provides resource isolation, lifecycle management, and virtual networking between VNF containers.
Virtual Network Function (VNF)	MediaMTX, droidgrid-recorder, droidgrid-api, analyser	Each containerised service implements one specific network function in software on shared hardware.
VNF Descriptor (VNFD)	docker-compose.yml service definitions	Formally describes compute, memory, network requirements of each VNF and its interconnect topology.
NFV Orchestrator (NFVO)	Docker Compose / Kubernetes controller	Instantiates, scales, migrates, and terminates VNF instances based on resource availability and policy.
VNF Manager (VNFM)	DroidGrid API service (Node.js)	Manages the operational lifecycle of individual VNFs: start/stop recording, configure streams, health checks.

ETSI NFV Concept	DroidGrid Equivalent	Notes
Service Function Chain (SFC)	Camera → Ingest → Decode → Analyse → Store → View	Ordered sequence of VNFs each stream traverses. Defined in software; reconfigurable without hardware changes.
Element Management System (EMS)	DroidGrid Pro Web UI (React)	Human-facing management interface for VNF configuration, monitoring, and control. Communicates via REST/WebSocket.

7.2 The Video Surveillance Service Function Chain

Service Function Chaining (SFC), defined in IETF RFC 7665, is the concept of routing traffic through an ordered sequence of network functions. In a traditional NVR appliance, the entire chain runs inside a single monolithic device: ingest, decode, motion detection, storage, and streaming are tightly coupled into proprietary firmware. NFV explicitly replaces this model by decomposing the chain into independent, composable VNFs. In DroidGrid Next-Gen, the surveillance service function chain has five stages, each implemented as a separately deployable VNF.



The critical NFV property here is composability: each VNF in the chain can be independently scaled, replaced, or bypassed. If motion detection is not required in a given deployment, VNF-4 is simply not instantiated and the chain routes directly from VNF-3 to VNF-5. If a different ML framework is preferred, VNF-4's container image is swapped without touching any other component. If recording capacity needs to increase, additional VNF-2 instances can be spawned across additional storage volumes. None of this is possible in a monolithic NVR appliance, where any change requires firmware replacement or hardware upgrade.

7.3 Elasticity, Isolation, and Fault Containment

Two of the most important operational properties that NFV provides — and that monolithic NVRs fundamentally cannot — are elastic scaling and fault containment.

Elastic scaling means that individual VNFs can have their resource allocations adjusted at runtime based on current demand. During an active recording session with ten cameras, VNF-2 (Recorder) needs more disk I/O and memory; VNF-4 (Analyser) may be processing a burst of motion events. Between sessions, both can be scaled down to minimal resource reservations. The ETSI NFV framework defines this via resource descriptors in the VNFD; in Docker Compose, the equivalent is the `deploy.resources` block:

```
vnf-recorder: deploy: resources: limits: { cpus: '2.0', memory: 1024M }
reservations: { cpus: '0.5', memory: 256M } vnf-analyser: deploy: resources:
limits: { cpus: '4.0', memory: 3G } reservations: { cpus: '0.1', memory: 512M }
```

Fault containment is equally important. In the current single-process DroidGrid, any unhandled exception in any module — a bad frame from one camera, a full disk on write, a segfault in OpenCV — risks corrupting the recording session for all cameras simultaneously. In the VNF model, each function runs in an isolated container with its own process space and memory. If VNF-4 (Analyser) crashes due to a model inference error, Docker restarts it within seconds while VNF-1 (Broker), VNF-2 (Recorder), and VNF-5 (Distributor) continue operating without interruption. Cameras keep recording. The live view keeps streaming. The crash is contained and logged.

7.4 VNF Descriptor Summary

A VNF Descriptor (VNFD) formally specifies a VNF's identity, requirements, and interfaces. The ETSI NFV standard defines VNFD using TOSCA YAML. In the DroidGrid context, the functional equivalent is the service definition inside docker-compose.yml. The table below gives a concise VNFD summary for each VNF in the proposed architecture, translating ETSI field names to their Docker Compose equivalents.

VNF ID	Image / Runtime	CPU Limit	RAM Limit	Exposed Ports	Chain Position
vnf-ingest (MediaMTX)	bluenviron/mediamtx:latest	0.5 core	128 MB	8554 RTSP 8888 HLS 8889 WebRTC	Stage 1 — source
vnf-recorder (FFMPEG)	droidgrid/recorder:2.x	1.5 cores	512 MB	internal only	Stage 2 — storage
vnf-decoder (Python + NVDEC)	droidgrid/decoder:2.x	0–2 cores (+GPU)	1 GB	internal only	Stage 3 — decode
vnf-analyser (optional)	droidgrid/analyser:2.x	0–4 cores (+GPU)	2 GB	internal only	Stage 4 — analysis
vnf-distributor (MediaMTX egress)	bluenviron/mediamtx:latest	0.5 core	128 MB	8889 WebRTC 8888 HLS	Stage 5 — delivery
vnf-api (Node.js)	droidgrid/api:2.x	0.5 core	256 MB	3000 REST/WS	Control plane
vnf-ui (React/Vite)	droidgrid/ui:2.x	0.2 core	128 MB	80 HTTP	Management (EMS)

7.5 NFV vs. Physical NVR: Decision Matrix

The table below directly compares a traditional physical NVR appliance against the proposed VNF deployment across the practical dimensions that matter for the DroidGrid use case. The NFV approach wins on every dimension where general-purpose hardware is already available — the only trade-off is higher initial engineering effort, which is a one-time cost versus a recurring hardware refresh cycle for the appliance model.

Dimension	Physical NVR Appliance	DroidGrid VNF Architecture
Camera protocol support	Fixed at manufacture (usually H.264 RTSP + ONVIF only)	Any protocol via VNF image swap; MJPEG, RTSP, WebRTC, HLS all supported
Camera count ceiling	Hard limit by port count: 4 / 8 / 16 / 32 SKUs	Soft limit by compute; scales horizontally by adding nodes
Hardware dependency	Vendor-specific embedded ARM SoC; non-replaceable	Any x86 or ARM Linux machine; cloud VM; Raspberry Pi cluster
Codec flexibility	Firmware-defined; upgrade requires vendor update	FFMPEG in container supports every codec; update = docker pull
AI / analytics integration	Optional paid proprietary module; limited model support	Any Python ML framework; swap VNF-4 image with zero downtime
Remote access model	Vendor P2P cloud relay; account required; privacy concerns	Self-hosted WebRTC via MediaMTX; no external dependency
Failure recovery time	Full appliance replacement; hours to days	Container auto-restart: under 5 seconds; data intact
Fault blast radius	Full system failure on firmware crash	Per-VNF isolation; one crash does not affect other VNFs
Upgrade path	New hardware purchase; data migration required	docker pull new_image && docker-compose up -d
Cost at 10 cameras	€300–€1500 dedicated hardware appliance	€0 additional hardware (existing machine)
ONVIF / PTZ support	Yes, built-in	Yes, via ONVIF VNF addition (Phase 4 roadmap)
Storage policy control	Limited; vendor-defined retention options	Fully programmable: size, age, event-based, tiered NAS

Core NFV Conclusion

The physical NVR is a hardware appliance performing a small, fixed set of network functions: ingest, transcode, record, stream, and manage cameras. NFV says: implement those functions in software and run them on commodity hardware. The proposed DroidGrid architecture is precisely this. The 'network' is the camera transport fabric; the 'functions' are the five VNF stages of the service chain; the 'virtualisation' is Docker on a general-purpose machine. Every ETSI NFV design principle — function isolation, elastic scaling, hardware independence, composable service chaining, and programmatic lifecycle management — is satisfied by the proposed design with zero proprietary hardware required.

■ SECTION 8

Implementation Roadmap

The migration from the current architecture to the proposed one does not require a complete rewrite. It can be executed in three phases over approximately 8–12 weeks, with each phase delivering independent value.

Phase 1 — Protocol Bridge (Weeks 1–2)

Immediate FPS fix without rewriting recording or UI logic.

- Install MediaMTX as a local Docker container.
- Configure it to pull MJPEG streams from DroidCam phones.
- Update DroidGrid's OpenCV VideoCapture URLs to point to MediaMTX's internal RTSP output instead of directly to phones.
- Result: MediaMTX maintains camera connections; Python only decodes frames needed for display; recording load is unchanged.
- Expected outcome: 30–50% FPS improvement at 5+ cameras, zero other changes.

Phase 2 — FFMPEG Recorder (Weeks 3–5)

Remove recording from Python entirely; cut CPU by ~60%.

- Add MediaMTX's built-in record path feature (record: yes in mediamtx.yml).
- MediaMTX writes H.264 directly to disk — no Python, no decode, no re-encode.
- Remove VideoWriter logic from droidgrid.py (now handled by MediaMTX).
- Add file index API to droidgrid-api: list recordings, delete, download.
- Result: recording becomes effectively free in CPU terms.

Phase 3 — WebRTC Live View in Browser (Weeks 6–8)

Embed live camera feeds in DroidGrid Pro web UI.

- MediaMTX already exposes a WebRTC endpoint at `http://localhost:8889/camname`.
- Embed WebRTC elements in the React camera grid via MediaMTX's built-in WHEP (WebRTC-HTTP Egress Protocol) API.
- Remove OpenCV imshow window — all viewing moves to the browser.
- Add motion detection notification via MediaMTX webhook → DroidGrid API.
- Result: full browser-based multi-camera view, no desktop window needed.

Phase 4 — IP Camera & ONVIF Support (Weeks 9–12)

Extend beyond DroidCam to professional IP cameras and NVR replacement.

- Add ONVIF camera discovery to droidgrid-api using `onvif-python` library.

- Auto-populate camera list from ONVIF device scan on local subnet.
- Support RTSP streams natively (IP cameras, Hikvision, Dahua, etc.).
- Add PTZ control UI for cameras that support it via ONVIF.
- Add retention policy: auto-delete recordings older than N days.
- Result: DroidGrid becomes a full software-defined NVR replacement.

Phase	Duration	Key Deliverable	FPS Impact	Complexity
1 — Protocol Bridge	1–2 weeks	MediaMTX as RTSP broker	+30–50%	Low
2 — FFMPEG Recorder	3–5 weeks	CPU-free pass-through recording	+60%	Medium
3 — WebRTC UI	6–8 weeks	Browser-based live grid view	N/A	Medium
4 — IP Cam / ONVIF	9–12 weeks	Full software-defined NVR	N/A	High

■ SECTION 9

Conclusion

DroidGrid's FPS degradation beyond four cameras is a structural consequence of MJPEG's per-frame decode cost combined with Python's single-process threading model. Reducing resolution offers diminishing returns because the dominant cost — JPEG entropy decoding — scales sub-linearly with pixel count.

The proposed architecture resolves this through three compounding changes. First, introducing a Go-based RTSP broker (MediaMTX) that maintains camera connections independently of the Python application. Second, moving recording to a pass-through path that copies H.264 bitstreams without decoding. Third, routing all live viewing through WebRTC — both eliminating OpenCV's imshow overhead and enabling browser-based access.

Together these changes transform DroidGrid from a Python script that decodes every frame of every camera into a proper multi-tier streaming system — architecturally similar to Shinobi CE and Frigate, but purpose-built for the DroidCam phone-as-camera use case and extending naturally to professional IP cameras via ONVIF.

The migration can be executed in four sequential phases with no downtime and no complete rewrite. Each phase delivers independent value. Phase 1 alone is estimated to double the camera count the current hardware can support, requiring only a Docker container install and a URL change in droidgrid.py.

Phase 1 2 weeks to first improvement	10+ Cameras at full FPS (final)	~0% Recording CPU (pass-through)	1 binary MediaMTX: no extra infra
--	---	--	---

This proposal was prepared as part of the Vision-Orchestration initiative. All architectural recommendations are based on analysis of the current DroidGrid codebase, public benchmarks from the open-source projects cited, and peer-reviewed research on low-latency video streaming architectures.

References: Frigate NVR (github.com/blakeblackshear/frigate) · MediaMTX (github.com/bluenvoron/mediamtx) · Moonfire NVR (github.com/scottlamb/moonfire-nvr) · Shinobi CE (gitlab.com/Shinobi-Systems/Shinobi) · IMS Surveillance Paper (MDPI Computers 2025, doi:10.3390/computers15010062) · NVIDIA DeepStream Edge-to-Cloud (docs.nvidia.com/mms) · Wowza RTSP-to-WebRTC Guide (wowza.com/blog) · OpenCV Multithreading Analysis (nrsyed.com, 2018)